

UNIT 4: TREE

4.1 Binary Tree

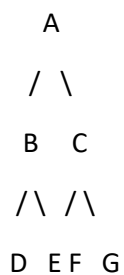
A **Binary Tree** is a non-linear data structure in which each node can have **at most two children**.

These two children are called:

1. **Left Child**
2. **Right Child**

A binary tree starts with a node called the **Root**.

Example



In this tree:

- **A** is the root node.
- **B** and **C** are children of **A**.
- **D** and **E** are children of **B**.
- **F** and **G** are children of **C**.
- **D**, **E**, **F**, and **G** are leaf nodes.

Characteristics of Binary Tree

- Each node has at most two children.
- Children are identified as left and right.
- The first node is called the root.
- A node may have zero, one, or two children.
- Binary trees are hierarchical data structures.

Applications of Binary Trees

Binary trees are used in:

- Searching algorithms.
- Sorting algorithms.
- Expression evaluation.
- File systems.

- Database indexing.
 - Decision-making systems.
 - Compiler design.
 - Hierarchical data representation.
-

4.2 Terminology of Binary Tree

The following terms are commonly used in binary trees.

1. Node

A **Node** is an individual element of a tree that contains data.

Example:

A

A is a node.

2. Root

The **Root** is the topmost node of a tree.

In the following tree:

A
/
B C

A is the root.

A tree has only one root node.

3. Parent

A node that has one or more children is called a **Parent Node**.

In the tree:

A
/
B C

A is the parent of B and C.

4. Child

A node directly connected below another node is called its **Child**.

In the above example:

- B is a child of A.
 - C is a child of A.
-

5. Sibling

Nodes having the same parent are called **Siblings**.

Example:

```
A
 / \
B   C
```

B and C are siblings.

6. Leaf Node

A node that has **no children** is called a **Leaf Node** or **Terminal Node**.

Example:

```
A
 / \
B   C
```

B and C are leaf nodes.

7. Internal Node

A node having at least one child is called an **Internal Node** or **Non-Leaf Node**.

In the above example, A is an internal node.

8. Edge

An **Edge** is the connection between two nodes.

Example:

```
A
 /
B
```

The connection between A and B is an edge.

9. Path

A **Path** is a sequence of nodes connected by edges.

Example:

A
/
B
/
D

The path from A to D is:

A → B → D

10. Degree of a Node

The **Degree** of a node is the number of children it has.

In a binary tree, the degree of a node can be:

- 0
 - 1
 - 2
-

11. Level

The position of a node in a tree is called its **Level**.

Usually:

- Root is at Level 0 or Level 1, depending on the convention.
- Its children are at the next level.

Example:

A Level 0
/ \
B C Level 1
/\ /\
D E F G Level 2

12. Height

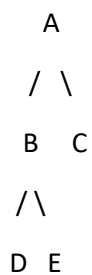
The **Height** of a tree is the length of the longest path from the root to a leaf.

For the above tree, the height is **2 edges** if the root is considered at level 0.

13. Subtree

A **Subtree** is a smaller tree formed from a node and all its descendants.

Example:



The subtree rooted at B is:



4.3 Types of Binary Tree

There are several types of binary trees.

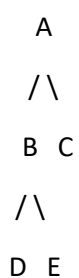
1. Full Binary Tree

A **Full Binary Tree** is a binary tree in which every node has either:

- 0 children, or
- 2 children.

No node has exactly one child.

Example:



2. Complete Binary Tree

A **Complete Binary Tree** is a binary tree in which:

- All levels except possibly the last are completely filled.
- The last level is filled from left to right.

Example:

```
A
 / \
B   C
 / \ /
D  E F
```

3. Perfect Binary Tree

A **Perfect Binary Tree** is a binary tree in which:

- Every internal node has exactly two children.
- All leaf nodes are at the same level.

Example:

```
A
 / \
B   C
 / \ / \
D  E F  G
```

4. Balanced Binary Tree

A **Balanced Binary Tree** is a tree in which the heights of the left and right subtrees are approximately balanced.

A balanced tree helps maintain efficient searching and other operations.

5. Degenerate Binary Tree

A **Degenerate Binary Tree** is a tree in which every parent node has only one child.

Example:

A
\
B
\
C
\
D

It behaves similarly to a linked list.

6. Skewed Binary Tree

A skewed binary tree is a special type of degenerate tree.

Left-Skewed Tree

All nodes have only left children.

A
/
B
/
C
/
D

Right-Skewed Tree

All nodes have only right children.

A
\
B
\
C
\
D

4.4 Traversing of Binary Tree

Tree Traversal is the process of visiting every node of a tree exactly once in a particular order.

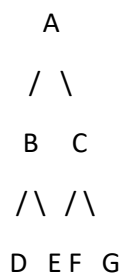
The three main depth-first traversal methods are:

1. Inorder Traversal
2. Preorder Traversal
3. Postorder Traversal

Another important method is:

4. Level Order Traversal

Consider the following binary tree:



1. Inorder Traversal

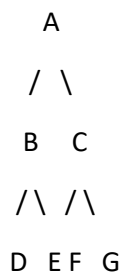
In **Inorder Traversal**, nodes are visited in the following order:

Left → Root → Right

Process

1. Traverse the left subtree.
2. Visit the root.
3. Traverse the right subtree.

For the given tree:



The traversal is:

D → B → E → A → F → C → G

Use

Inorder traversal of a **Binary Search Tree (BST)** produces elements in sorted order.

2. Preorder Traversal

In **Preorder Traversal**, nodes are visited in the following order:

Root → Left → Right

Process

1. Visit the root.
2. Traverse the left subtree.
3. Traverse the right subtree.

For the given tree:

A → B → D → E → C → F → G

Applications

- Creating a copy of a tree.
- Prefix expression generation.
- Tree serialization.

3. Postorder Traversal

In **Postorder Traversal**, nodes are visited in the following order:

Left → Right → Root

Process

1. Traverse the left subtree.
2. Traverse the right subtree.
3. Visit the root.

For the given tree:

D → E → B → F → G → C → A

Applications

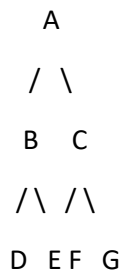
- Deleting a tree.
- Postfix expression generation.
- Evaluating expression trees.

4. Level Order Traversal

In **Level Order Traversal**, nodes are visited level by level from top to bottom.

It generally uses a **Queue** data structure.

For the given tree:



The traversal is:

A → B → C → D → E → F → G

Comparison of Tree Traversals

Traversal	Order	Result
Inorder	Left → Root → Right	D B E A F C G
Preorder	Root → Left → Right	A B D E C F G
Postorder	Left → Right → Root	D E B F G C A
Level Order	Level by Level	A B C D E F G

headnullnullAnextBnextXnextCnext

Outside the list

X → nullStep 1 of 4

Insert X after B: Initialize X.

InsertDelete

InsertDelete

Step

Step

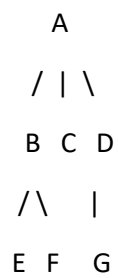
Note: The visualization above is for linked-list operations and is not specifically a binary-tree traversal visualizer. The traversal sequences given above are the correct sequences for the example binary tree.

4.5 General Tree

A **General Tree** is a non-linear hierarchical data structure in which a node can have **any number of children**.

Unlike a binary tree, a node in a general tree is not restricted to a maximum of two children.

Example



In this tree:

- A is the root.
- A has three children: B, C, and D.
- B has two children: E and F.
- D has one child: G.

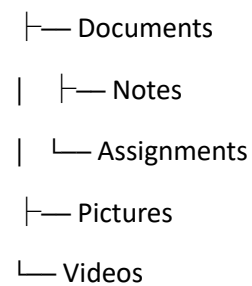
Characteristics of General Tree

- A node can have zero or more children.
- The tree has one root node.
- Nodes are connected using edges.
- There are no cycles.
- It represents hierarchical relationships.

Examples of General Trees

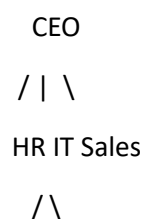
1. File System

Computer



A folder can contain multiple files and subfolders.

2. Organization Structure



3. HTML Document Structure

HTML documents can also be represented using a tree-like structure called the **Document Object Model (DOM)**.

Difference Between Binary Tree and General Tree

Feature	Binary Tree	General Tree
Maximum Children	2	Any number
Children Names	Left and Right	No fixed names
Structure	More restricted	More flexible
Example	Expression tree	File system
Traversal	Inorder, Preorder, Postorder, Level Order	Preorder, Postorder, Level Order

Applications of General Trees

General trees are used in:

- File and folder systems.
- Organization hierarchies.
- HTML/XML document structures.
- Database systems.
- Artificial intelligence.
- Game development.
- Computer networks.

Conclusion

A **Tree** is a non-linear hierarchical data structure used to represent relationships between data elements. A **Binary Tree** allows each node to have a maximum of two children, while a **General Tree** allows a node to have any number of children. Important binary-tree concepts include root, parent, child, leaf, edge, level, height, and subtree. Binary trees can be classified into full, complete, perfect, balanced, degenerate, and skewed trees. Tree traversal methods such as **Inorder, Preorder, Postorder, and Level Order** are used to visit all nodes in different sequences.